Visualizing Geodata

•••

POLI3148 Data Science in Politics and Public Administration

Dr. Chen Haohan

Agenda

- Logistics
  - Text Analysis readings
  - Assignment 1 (20%)
  - Final Project
    - Bonus project proposal due soon
    - Discussions about project ideas are welcome
- Geodata Visualization

Why geodata visualization?

Understand spatial distribution of political data of interest, for example:

- Economy
- Votes
- Protests
- Crime
- Conflicts

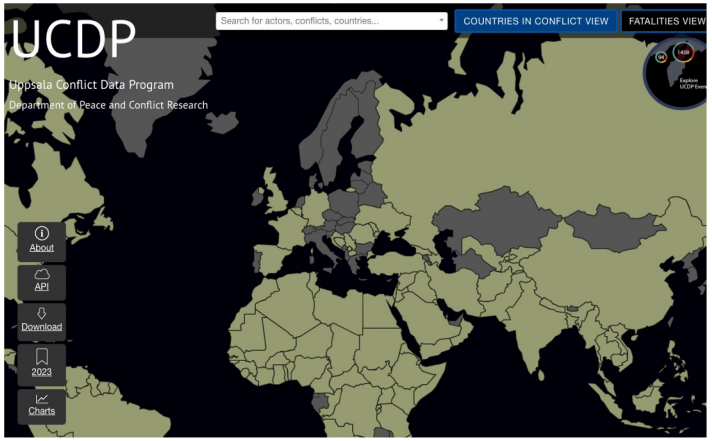
Python Library: GeoPandas

https://geopandas.org/en/latest/index.html

“GeoPandas is an open source project to add support for geographic data to **pandas** objects.”

Cases for Demonstration

- V-Dem: https://www.v-dem.net/
- UCDP Conflict data: https://ucdp.uu.se/



Map skills into our data science workflow

- **Data Input**
  - “Map”
  - Data with geo-referenced named locations (e.g., countries, cities, counties)
  - Data with geo-referenced points
- **Data processing**
  - Merge data with the map
  - Aggregate
- **Data Output:** Save and export the plot

Map skills into our data science workflow

- **Data Input**
  - "Map"
  - Data with geo-referenced named locations (e.g., countries, cities, counties)
  - Data with geo-referenced points
- **Data processing**
  - Merge data with the map
  - Aggregate
- **Data Output:** Save and export the plot

Data Input: Maps

- Maps show up as “colored shapes” in our mind
- How about for Python (and, more generally, computers)?

*In a simplest form, maps are represented as **points** that carve the **boundaries** of geographic units of interest.*

Don’t get it? An example will help.

Data Input: Maps

Download a map readable by Python

https://www.naturalearthdata.com/downloads/10m-cultural-vectors/

Admin 0 – Countries

*There are **258 countries** in the world. Greenland as separate from Denmark. Most users will want this file instead of sovereign states, though some users will want map units instead when needing to distinguish overseas regions of France.*

*Natural Earth shows **de facto** boundaries by default according to who controls the territory, versus de jure.*

Download countries (210.08 KB) version 5.1.1

Download without boundary lakes (212.3 KB) version 5.1.1

About | Issues | Version History »

Admin 0 – Countries point-of-views

*These optional point-of-view (POV) variants show the worldview for several dozen countries according to **de jure** boundaries (as prescribed by the home country's law and/or local conventions) and their global alliances. This results in **between 246 and 258 countries**, depending on who's counting, with variation in border alignments.*

Download countries (Argentina POV) (4.68 MB) version 5.1.1

Download countries (Bangladesh POV) (4.68 MB) version 5.1.1

Download countries (Brazil POV) (4.69 MB) version 5.1.1

Download countries (China POV) (4.68 MB) version 5.1.1

Download countries (Egypt POV) (4.69 MB) version 5.1.1

Download countries (France POV) (4.69 MB) version 5.1.1

What you have downloaded...

Information of the map stored in multiple files

The **.shp** file is the most important one.

Name	Date Modified	Size	Kind
ne_10m_admin_0_countries_chn.dbf	May 9, 2022 at 12:45	794 KB	Document
ne_10m_admin_0_countries_chn.prj	May 9, 2022 at 12:45	145 bytes	Document
ne_10m_admin_0_countries_chn.README.html	May 9, 2022 at 13:13	39 KB	HTML text
ne_10m_admin_0_countries_chn.shp	May 9, 2022 at 12:45	8.8 MB	Document
ne_10m_admin_0_countries_chn.shx	May 9, 2022 at 12:45	2 KB	Document
ne_10m_admin_0_countries_chn.VERSION.txt	May 9, 2022 at 13:13	7 bytes	Plain Text

Task 1: Draw An Empty World Map

Map skills into our data science workflow

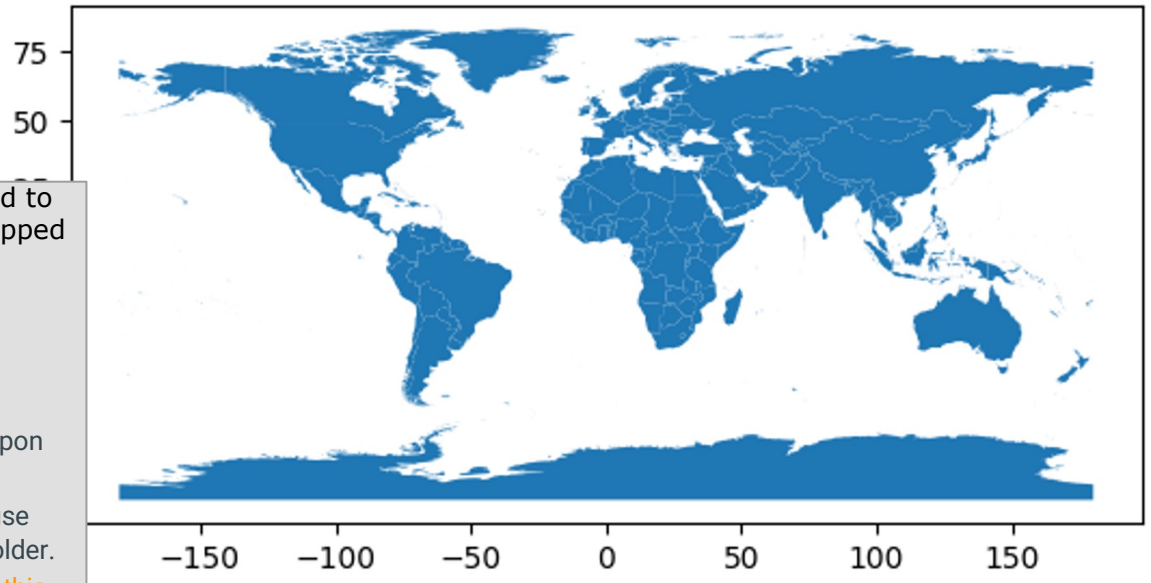
- **Data Input**
  - "Map"
  - Data with geo-referenced named locations (e.g., countries, cities, counties)
  - Data with geo-referenced points
- **Data processing**
  - Merge data with the map
  - Aggregate
- **Data Output:** Save and export the plot

Getting Started: Load required libraries

```
import pandas as pd

import geopandas as gpd

import matplotlib.pyplot as plt

import seaborn as sns
```

Load Map Data and Plot

```
world = gpd.read_file("[PATH TO YOUR FOLDER]/ne_10m_admin_0_countries_chn.zip")

world

world.plot()
```



Alternatively, you can unzip the folder, upload to Google Drive, and load the map from the unzipped folder. For example:

```
world = gpd.read_file("[PATH TO YOUR FOLDER]/ne_10m_admin_0_countries_chn")
```

If you use **Safari on Mac**, it will automatically unzip upon Download. Don't re-compress it -- there will be error reading a folder re-compressed folder by Mac, because Mac create an additional layer for the compressed folder. To stop Safari from automatically unzipping, refer to this tutorial.

How does the table turn into map?

THE column that makes a map!

```
world.head(5)
```

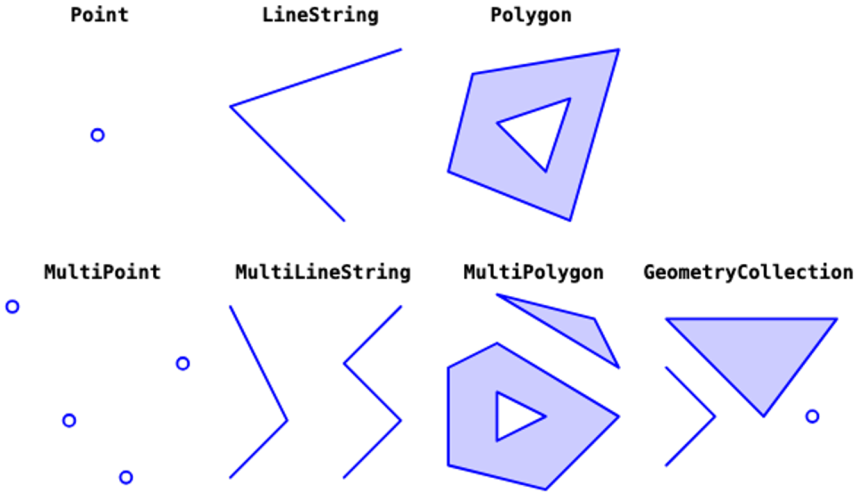
	ADM0_A3_CN	featurecla	scalerank	LABELRANK	SOVEREIGNT	SOV_A3	ADM0_D
0	IDN	Admin-0 country	0	2	Indonesia	IDN	
1	MYS	Admin-0 country	0	3	Malaysia	MYS	
2	CHL	Admin-0 country	0	2	Chile	CHL	
3	BOL	Admin-0 country	0	3	Bolivia	BOL	
4	PER	Admin-0 country	0	2	Peru	PER	

5 rows × 169 columns

FCLASS_SE	FCLASS_BD	FCLASS_UA	geometry
None	None	None	MULTIPOLYGON (((117.70361 4.16341, 117.70361 4...
None	None	None	MULTIPOLYGON (((117.70361 4.16341, 117.69711 4...
None	None	None	MULTIPOLYGON (((-69.51009 -17.50659, -69.50611...
None	None	None	POLYGON ((-69.51009 -17.50659, -69.51009 -17.5...
None	None	None	MULTIPOLYGON (((-69.51009 -17.50659, -69.63832...

What are in the *geometry* column

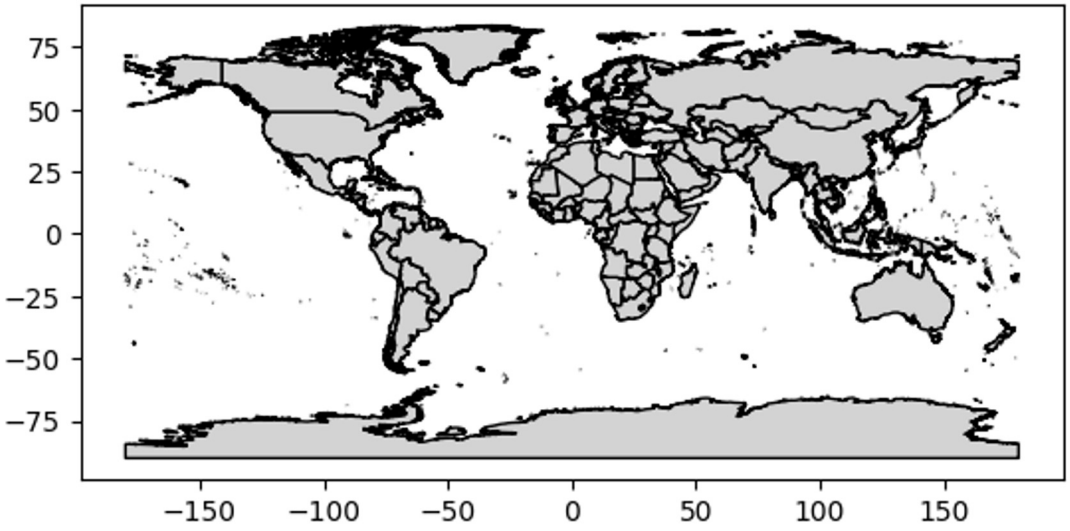
Some terminology concerning how geographic shapes are represented in Python



Type	WKT example
"Point"	POINT (30 10)
"LineString"	LINESTRING (30 10, 10 30, 40 40)
"Polygon"	POLYGON ((35 10, 45 45, 15 40, 10 20, 35 10),(20 30, 35 35, 30 20, 20 30))
"MultiPoint"	MULTIPOINT (10 40, 40 30, 20 20, 30 10)
"MultiLineString"	MULTILINESTRING ((10 10, 20 20, 10 40),(40 40, 30 30, 40 20, 30 10))
"MultiPolygon"	MULTIPOLYGON (((40 40, 20 45, 45 30, 40 40)),((20 35, 10 30, 10 10, 30 5, 45 20, 20 35),(30 20, 20 15, 20 25, 30 20)))
"GeometryCollection"	GEOMETRYCOLLECTION (POINT (40 10),LINESTRING (10 10, 20 20, 10 40),POLYGON ((40 40, 20 45, 45 30, 40 40)))

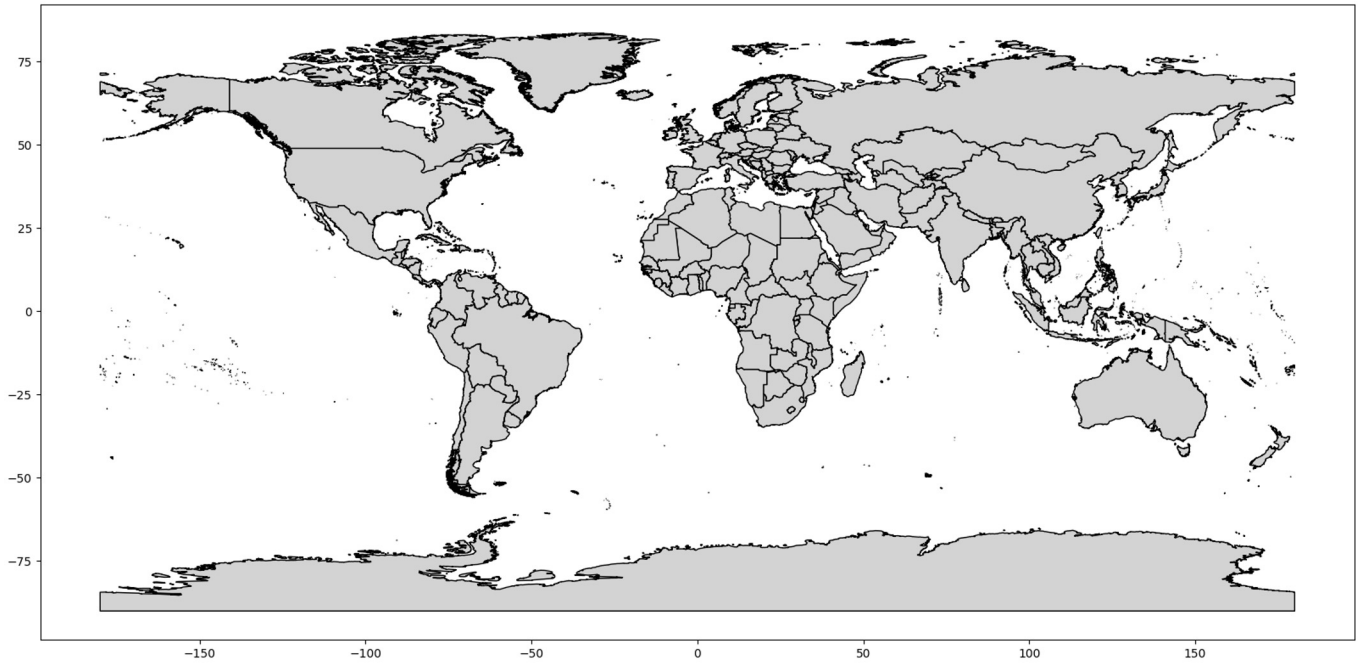
Prettify your map: Color

world.plot(color = "lightgray", edgecolor = "black")



Prettify your map: Size

world.plot(color = "lightgray", edgecolor = "black", figsize = (20, 10))



Try it yourself

- Can you add a title and remove the axes labels?
- What are the other columns in the “world” GeoPandas dataframe? What do they mean?
- Play with **figsize**. Is the size perfectly in line with your setting? If not, why? Hint: Check the “aspect” argument in the documentation https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.plot.html

Task 2: Draw a World Map of “Wealth and Health”

“Choropleth”

Map skills into our data science workflow

- **Data Input**
  - “Map”
  - **Data with geo-referenced named locations (e.g., countries, cities, counties)**
  - Data with geo-referenced points
- **Data processing**
  - **Merge data with the map**
  - Aggregate
- **Data Output:** Save and export the plot

Data with geo-referenced named locations

Datasets whose data points are identified with geo-referenced named locations

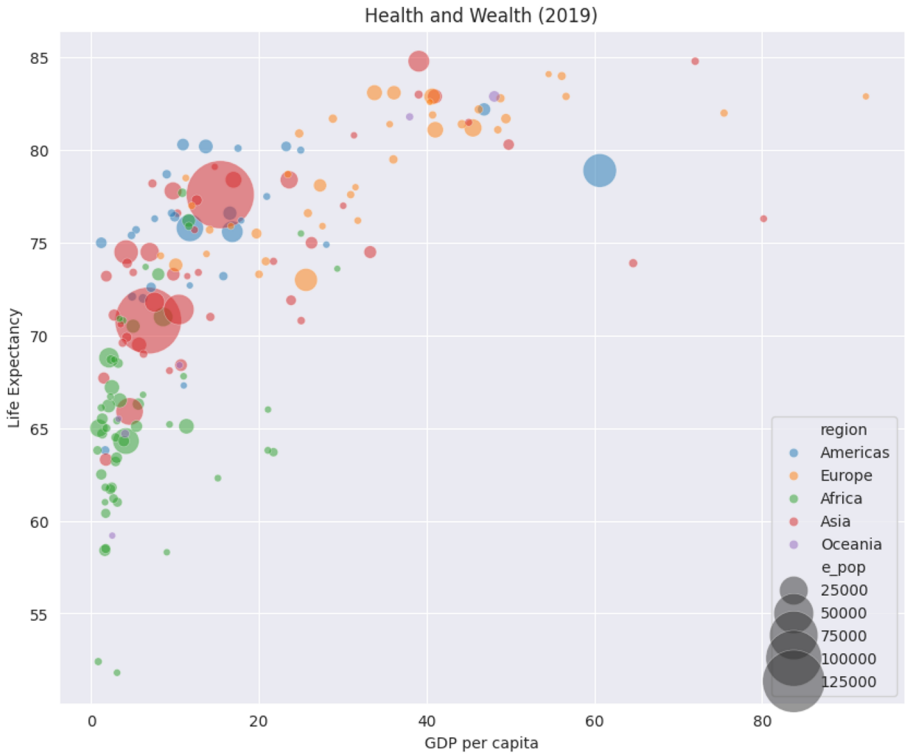
Example we use

- Dataset of GDP per capita across **countries**
- Dataset of life expectancy across **countries**

Other examples: All kinds of data aggregated at the level of …
Continents/ countries/ states/ provinces/ cities/ township/ zip code

Maps re the geo-distribution of health and wealth?

- We made the figure on the right-hanc side last time.
- What if we want to know the geographic distribution?



Load V-Dem Data

Re-use code from last lecture

df_vdem = pd.read_csv("[PATH TO VDEM DATA]/V-Dem-CY-Full+Others-v14.csv")

df_vdem_s = df_vdem.loc[df_vdem.year == 2019, :]

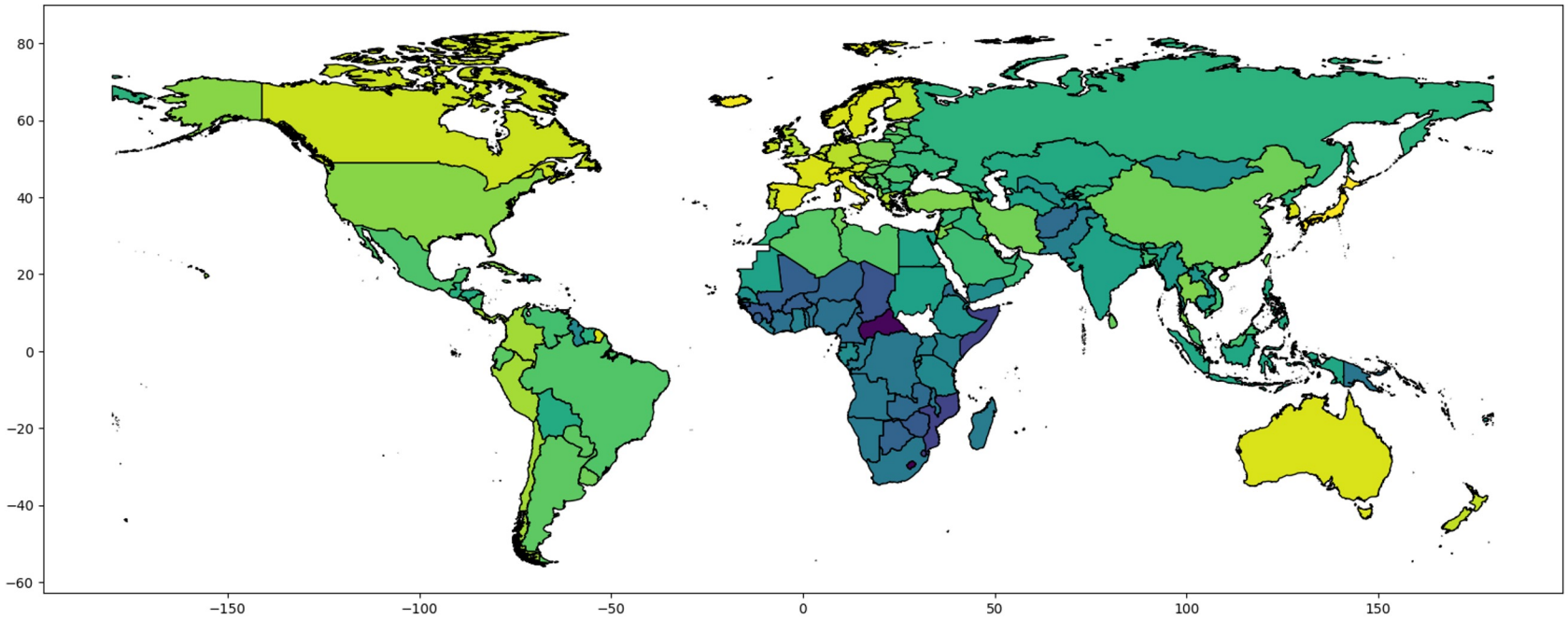
Merge Map Information with Data

world_vdem = pd.merge(world, df_vdem_s, left_on = "ADM0_A3_CN", right_on = "country_text_id", how = "left")

Why match on these two keys?

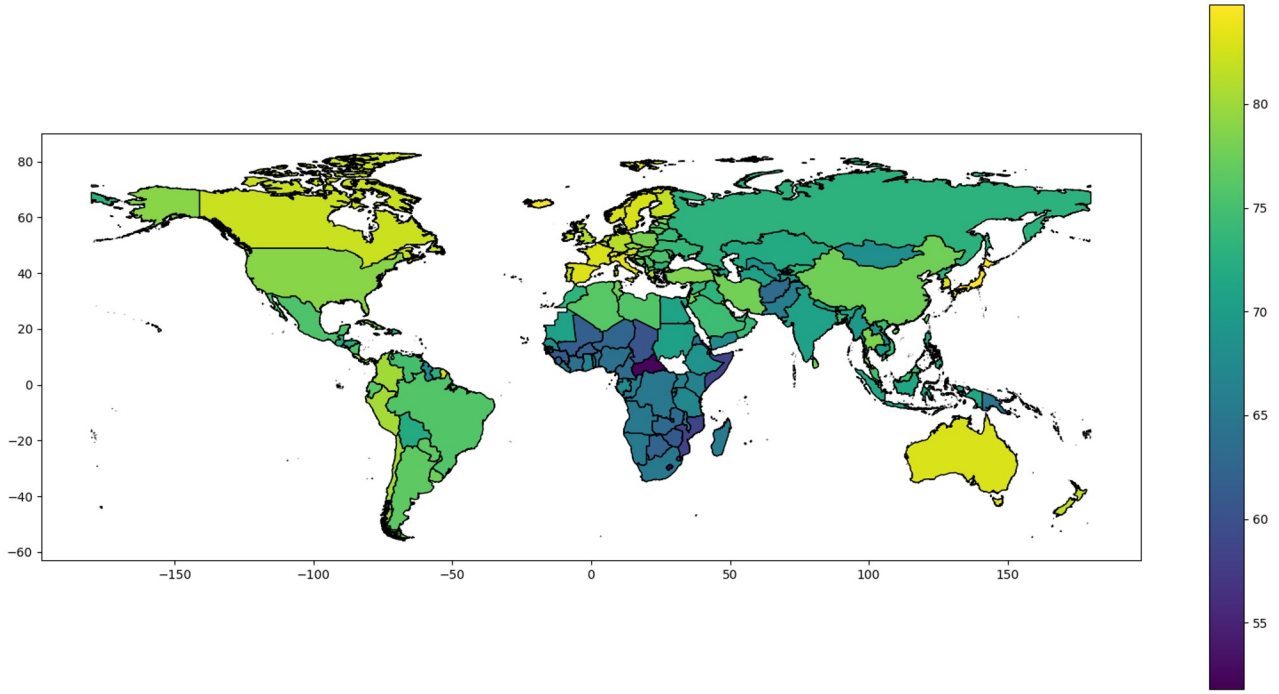
Plot Life Expectancy

world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10))



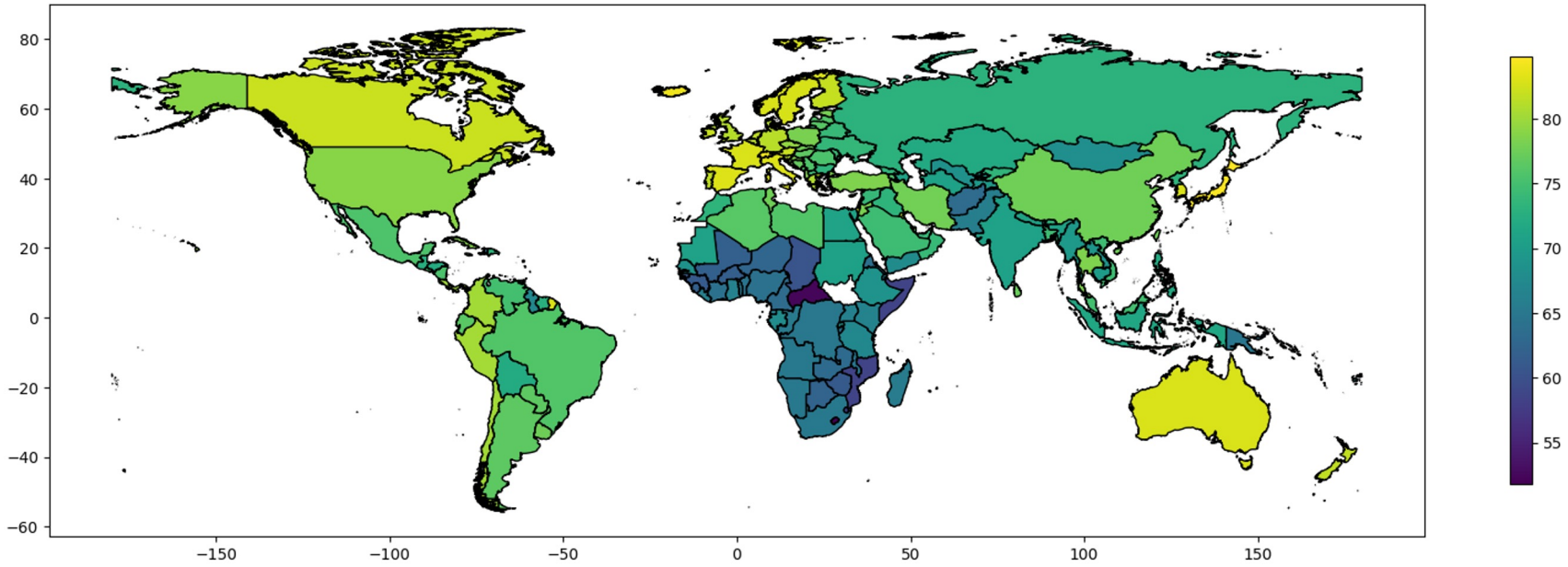
Important: Add Legends

world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10), legend = True)



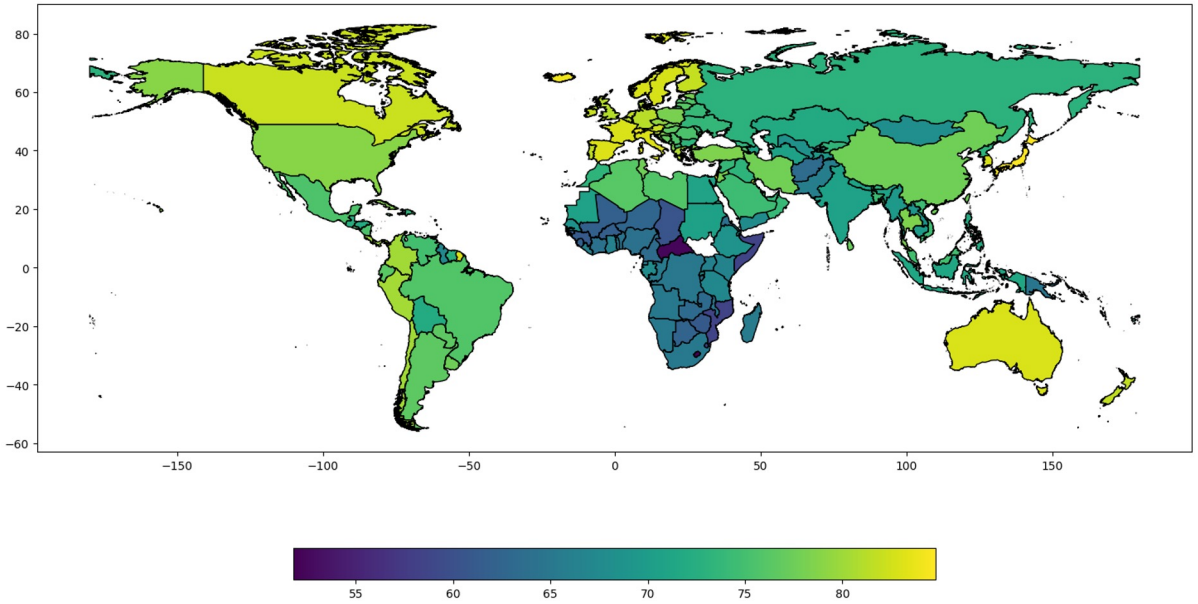
Pretty plot: Resize legend

world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10), legend = True, legend_kwds = {'shrink': 0.5})



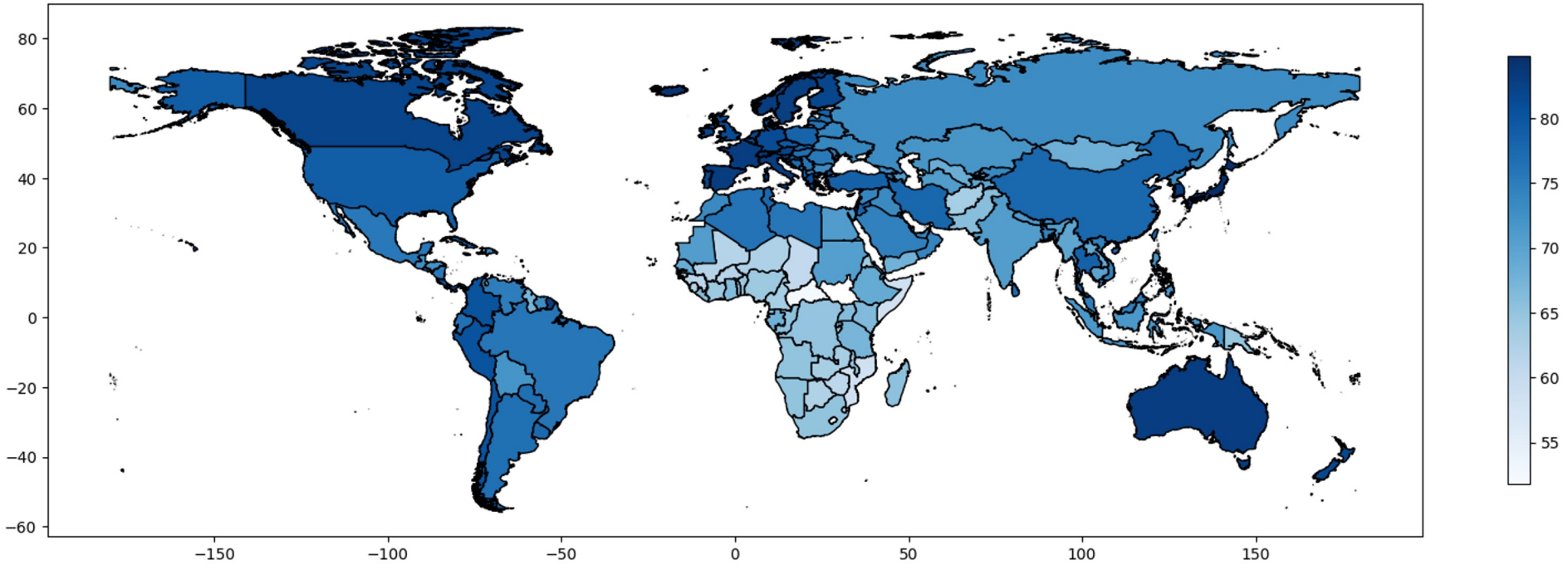
Pretty plot: Make legend horizontal (if you choose)

world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10), legend = True, legend_kwds = {"orientation": "horizontal", 'shrink': 0.5})



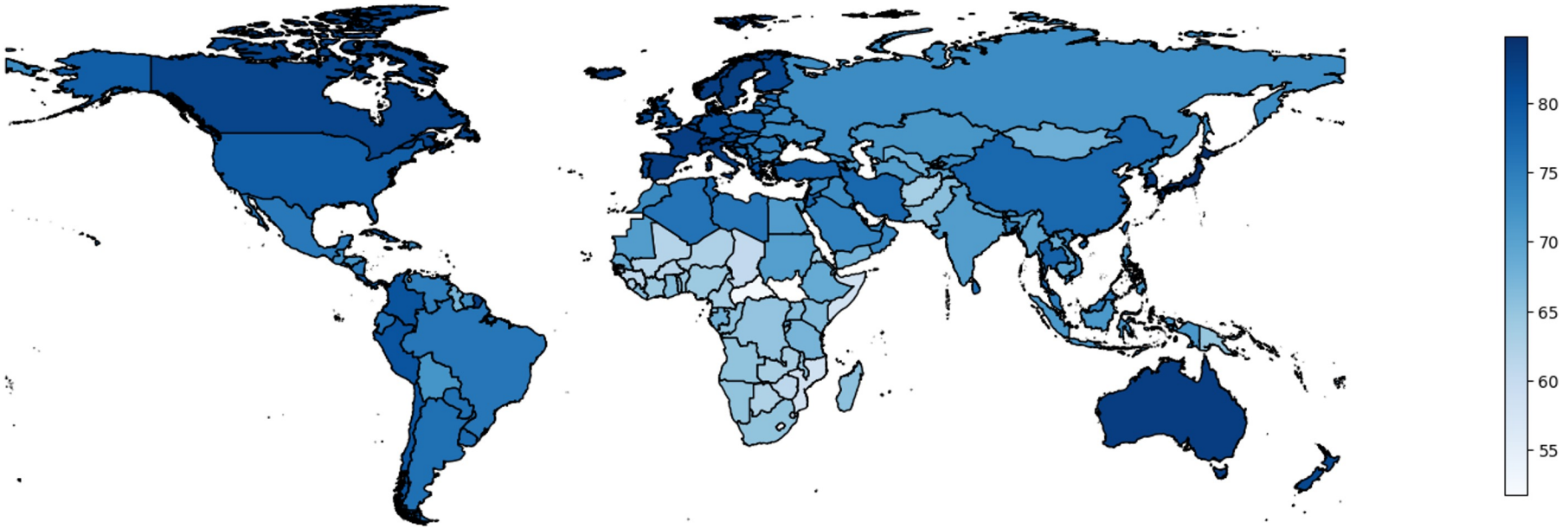
Prettify plot: Change color palette

world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10), legend = True, legend_kwds = {'shrink': 0.5}, cmap = "Blues")



Prettify plot: Remove axes

```
world_vdem.plot(column = "e_pelifeex", edgecolor = "black", figsize = (20, 10),
legend = True, legend_kwds = {'shrink': 0.5}, cmap = "Blues").set_axis_off()
```



Think and try it yourself

Try yourself

- Choose a color palette you like
- Remove the axes texts and labels of ALL maps
  Hint: "**.set_axis_off()**"
- Visualize the geo-distribution of the "wealth" indicator: GDP per capita
- Check the map: Find some white area? Why?

What are all the maps we have drawn? **Choropleths**

Wikipedia

A choropleth map is a type of statistical thematic map that uses pseudocolor, meaning color corresponding with an aggregate summary of a geographic characteristic within spatial enumeration units, such as population density or per-capita income.

Task 3: Visualize Global Distribution of Conflicts

“Bubble Map”

Map skills into our data science workflow

- **Data Input**
  - “Map”
  - Data with geo-referenced named locations (e.g., countries, cities, counties)
  - Data with geo-referenced points
- **Data processing**
  - Merge data with the map
  - Aggregate
- **Data Output:** Save and export the plot

Case: UCDP Georeferenced Event Data

- Download the UCDP GED dataset: https://ucdp.uu.se/downloads/ged/ged241-csv.zip
- Upload to your Google Drive project folder

UCDP Georeferenced Event Dataset (GED) Global version 24.1

This dataset is UCDP's most disaggregated dataset, covering individual events of organized violence (phenomena of lethal violence occurring at a given time and place). These events are sufficiently fine-grained to be geo-coded down to the level of individual villages, with temporal durations disaggregated to single, individual days.

Available as:

Please cite:

• Davies, Shawn, Garoun Engström, Therese Pettersson & Magnus Öberg (2024). Organized violence 1989-2023, and the prevalence of organized crime groups. Journal of Peace Research 61(4).

• Sundberg, Ralph and Erik Melander (2013) Introducing the UCDP Georeferenced Event Dataset. Journal of Peace Research 50(4).

Load UCDP-GED data

No need to unzip, to save space.

```
df_ucdp = pd.read_csv("[PATH TO DATA]/ged241-csv.zip")

df_ucdp.shape

df_ucdp.loc[1, :]

df_ucdp.loc[1, :].to_dict() # for more details
```

where_description	Kabul airport (Abbey gate entrance)
adm_1	Kabul province
adm_2	Kabul district
latitude	34.564444
longitude	69.217222
geom_wkt	POINT (69.2172222 34.5644444)
priogrid_gid	179779

```
'latitude': 34.564444,
'longitude': 69.217222,
'geom_wkt': 'POINT (69.2172222 34.5644444)',
'priogrid_gid': 179779,
'country': 'Afghanistan',
'country_id': 700,
```

Check one example row. Recommended operation to get a sense of the data

What is PRIOGRID_GID? Look it up!

Data with Geo-Referenced Points: Parse Geo information

The geo-location of each data point is represented by its longitude and latitude.

df_ucdp.geom_wkt # The geo-information was read as “object” (what is it?)

df_ucdp.loc[:, 'geometry'] = gpd.GeoSeries.from_wkt(df_ucdp.geom_wkt)

Parse the geo-information with the GeoPandas Library, then create a new column named “geometry” to store the parsed information

Check its data type!

Warning: difficult part. Re-read to make sure you understand what it does

Data with Geo-Referenced Points: Parse Geo information (alternative approach)

df_ucdp.loc[:, 'geometry'] = gpd.points_from_xy(df_ucdp.longitude, df_ucdp.latitude)

Use numeric **longitude** and **latitude** columns to create a geo-location indicator.

Note: Longitude should come first, then latitude.

Try what you get if you flip the two… I made this mistake before. You can make this mistake when you work with data from a single country/ continent.

Prepare the Geo-Referenced Data

df_ucdp_geo = gpd.GeoDataFrame(df_ucdp, crs = world_vdem.crs, geometry = df_ucdp.geometry)

What's this? (Warning: **Difficult part**. Please re-read to make sure you understand)

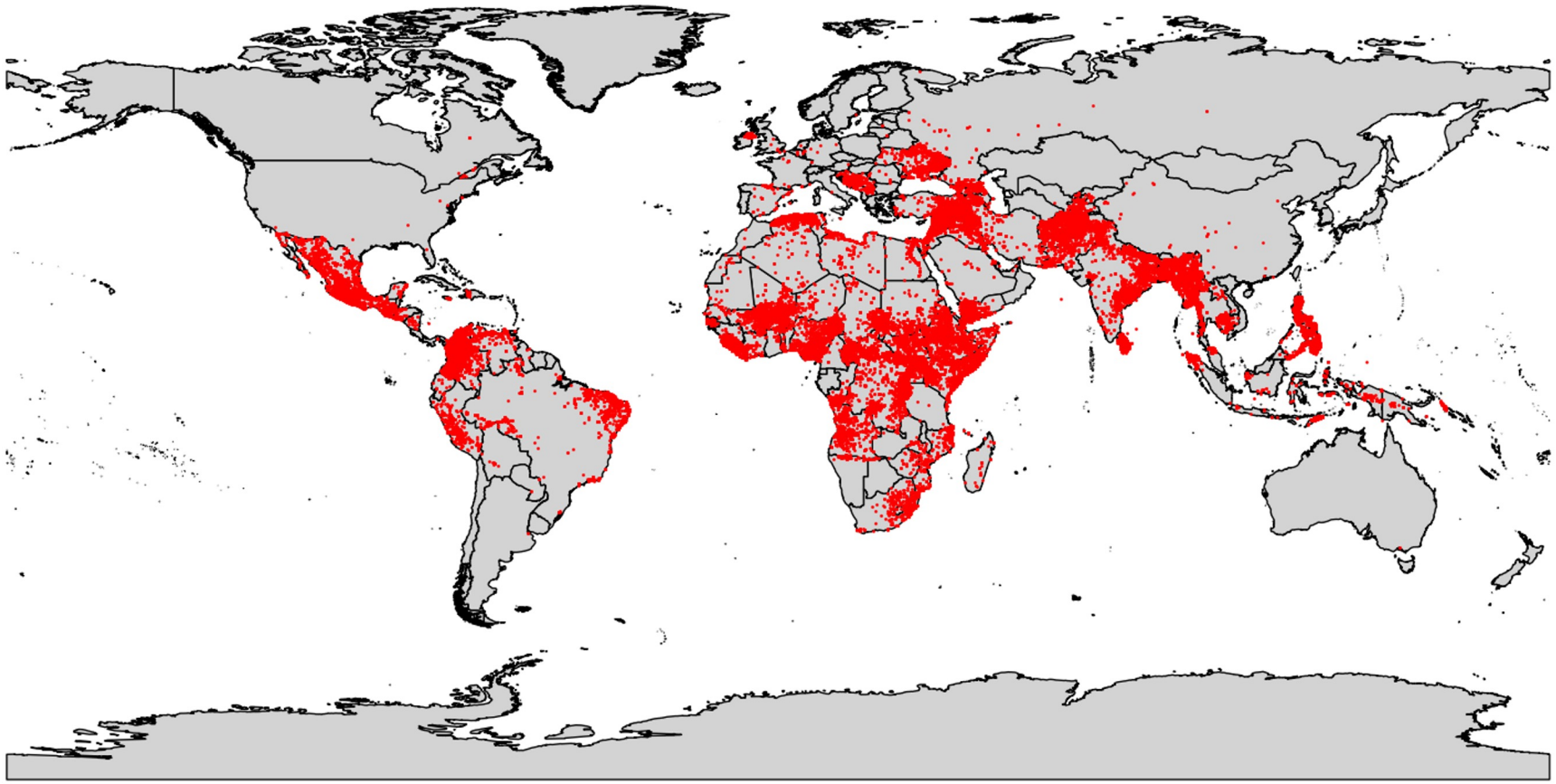
- **df_ucdp** is a **Pandas** dataframe, NOT a **GeoPandas** dataframe. The step convert it to a GeoPandas Dataframe
- Arguments
  - **crs:** Controls the projection system (see Kaggle reading). We use the same system as the map
  - **geometry:** Tell Python what makes the "geometry" column

Plot

```
ax = world_vdem.plot(color = "lightgray", edgecolor = "black", figsize = (20, 10))

df_ucdp_geo.plot(ax=ax, marker = 'o', color = 'red', markersize = 1).set_axis_off()
```

What is this?

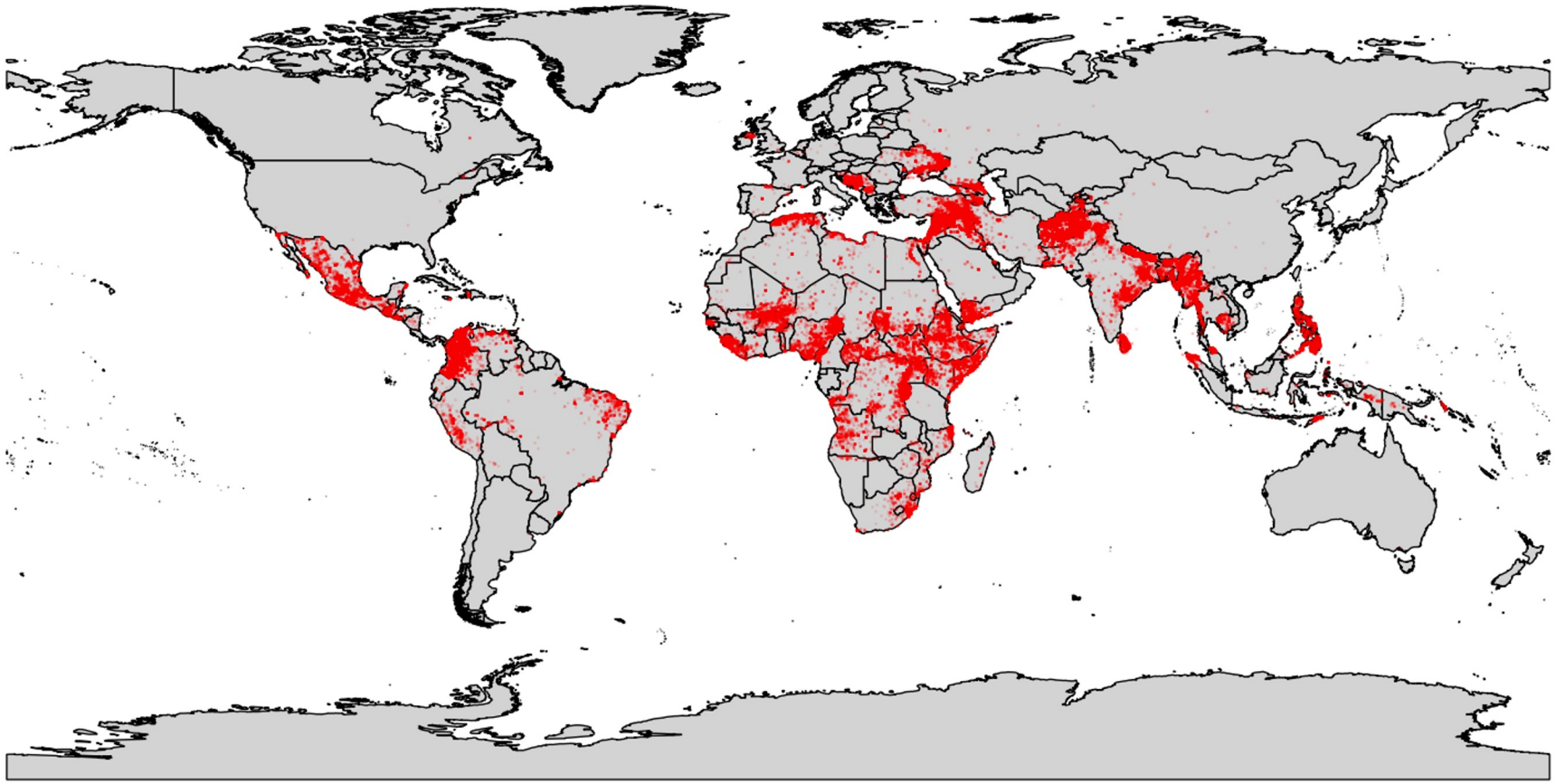
- You make a figure with the two layers
- The 1st layer is the world map, stored temporarily in an object named **ax**
- The 2nd layer include all the points indicating conflict incidents. Painted **on top of** the world map



Prettify your map: Set transparency

When you have lots of points, overlapping points can make your figure uninformative

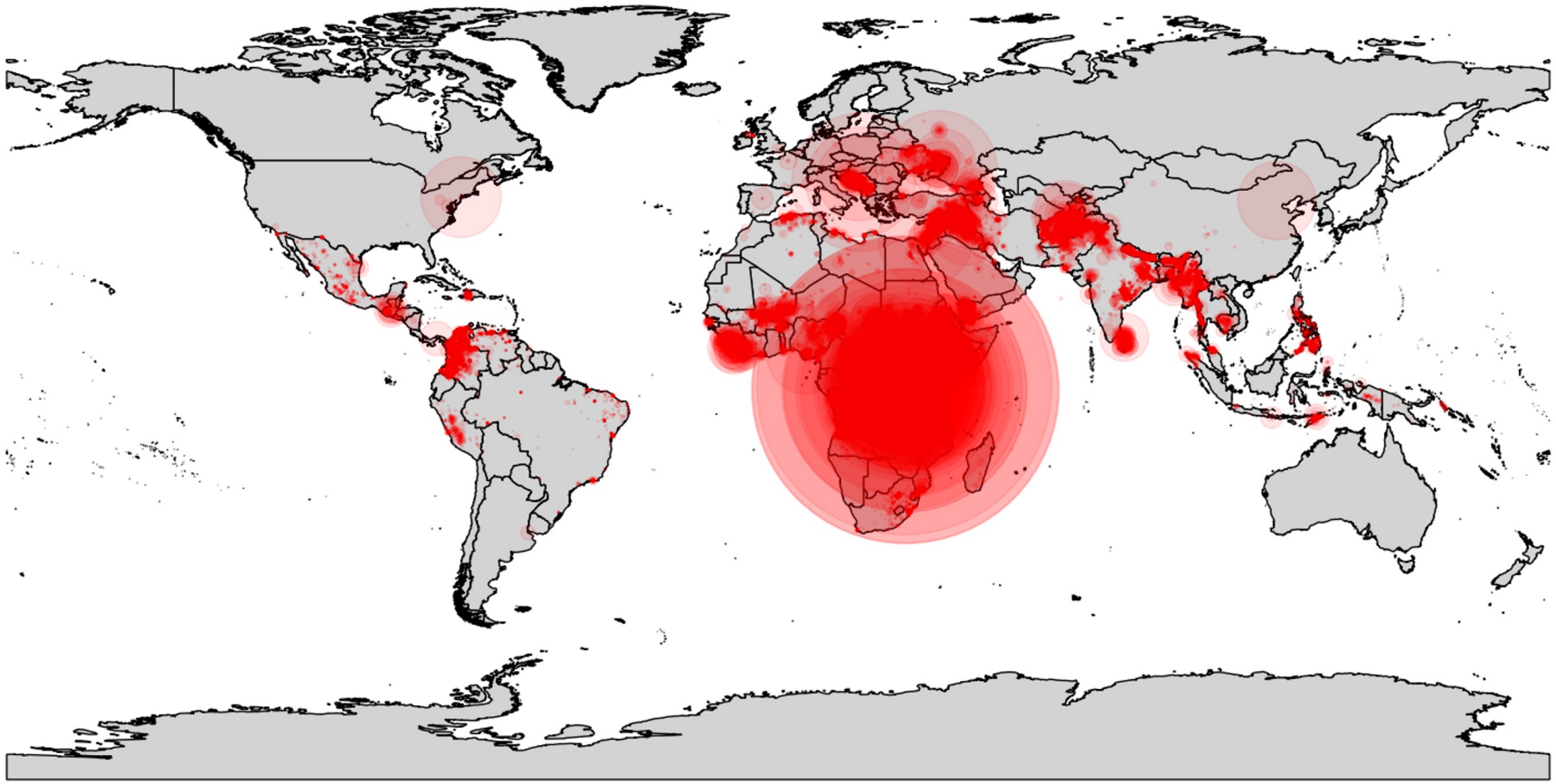
```
ax = world_vdem.plot(color = "lightgray", edgecolor = "black", figsize = (20, 10))

df_ucdp_geo.plot(ax=ax, marker = 'o', color = 'red', markersize = 1, alpha = 0.1).set_axis_off()
```



Make Figure More Informative: Sizes

```
ax = world_vdem.plot(color = "lightgray", edgecolor = "black", figsize = (20, 10))

df_ucdp_geo.plot(ax=ax, marker = 'o', color = 'red', markersize = 'deaths_civilians',
alpha = 0.1).set_axis_off()
```

Let sizes indicate deaths of civilians



Try yourself

- Add legend to the last figure (think: why do we want a legend?)
- Change the colors to make the maps prettier
- Resize the bubbles to make them more informative
  - Divide by a constant?
  - Square-root transformation?
- Add titles to the maps
- Visualize different variables related to the conflict

Summary

Today: Two types of data, two types of basic geo visualization

- Data associated with geo-referenced named locations -- Choropleths
- Data associated with geo-referenced points -- Bubble maps

The two basic types of geo data visualization. They are helpful because

- Straightforward from the data, no complicated data wrangling
- Wide range of application
  - Descriptive statistics
  - Exploratory analysis
  - Communicate output to the public

Note: Don’t get distracted and intimidated

Geodata visualization is a BIG industry

- Many tools are designed by geographers, environmental scientists etc.
- Many are legitimately complicated because of the complexity of application in these fields where the tools are developed
- However, our application, even for many advanced research, only need a tiny portion of them
- Don’t waste time to learn things you don’t need (or feeling bad about not knowing the complicated things). Just focus on what you need.

Other Geo-referenced datasets of conflict events

- ACLED: https://acleddata.com/ (more diverse types of conflicts)
- GDELT: https://www.gdeltproject.org/ (comprehensive, but more “noisy”)

When you use these event data, read the codebook carefully to understand how these events are defined.

Further Readings

GeoPandas documentation
https://geopandas.org/en/latest/index.html

Note: **GeoPandas has evolved.** Many resources you find online can be obsolete. Don't be surprised if they don't work because of recent updates. Generative AIs, typically trained with older data, can make the same mistakes.

Whenever you are unsure, consult with the official documentation (the link above)